

Octopus CXL Memory Pooling: RL vs. Greedy, Training System, and Next Steps

Pulak Mehrotra

April 2026

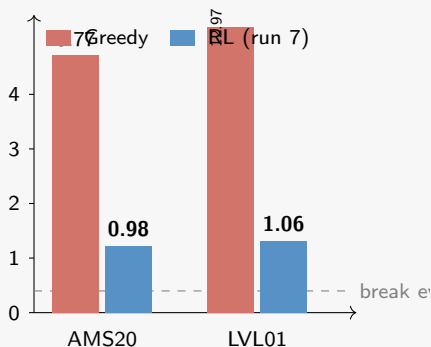
Three takeaways.

1. **RL beats greedy dramatically on two held-out traces.** The learned policy achieves near-break-even pooling savings while greedy needs 4–13 $\times$ more MPD capacity. The system generalises to unseen workloads.
2. **The training system is not the bottleneck.** Data augmentation across 10 traces, a redesigned reward, and 32 parallel environments eliminated the collapse failure and enabled generalisation.
3. **The remaining ceiling is a reward design problem.** Savings plateau at $\sim 1\text{--}2\%$ from 100k steps onward with no improvement over 2M steps. The agent learns to game the proxy reward rather than maximise true pooling savings.

RL outperforms greedy on both held-out test traces.

Pooling ratio (lower = better;
1.0 = break even)

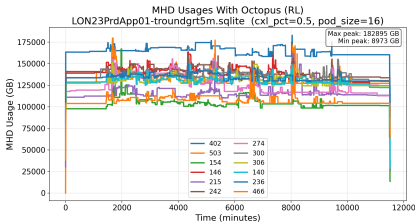
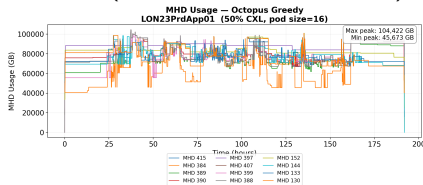
What this means



- ▶ Pooling ratio < 1 : fewer MPDs needed than a non-pooled baseline.
- ▶ Greedy needs $3.77\times$ the capacity on AMS20 and $12.97\times$ on LVL01 because it piles load onto the same MPDs.
- ▶ RL (run 7) is at 0.98 and 1.06 — near break-even on both traces it has never seen.
- ▶ $n = 50$ **random pod mappings** per data point (error bars: std ≈ 0.02 – 0.22).

Greedy packs load onto a few MPDs; RL spreads it.

Greedy (least-loaded MPD first) RL (run 7, reward A)



A few MPDs hit the ceiling; most are idle.

Load spread across MPDs; no single bottleneck.

LON23 trace (validation), CXL fraction 0.5, pod of 16 nodes. Each line = one MPD's load over time.

Two independent improvements were needed to get here.

1. Data: augmentation across 10 traces

- ▶ **Before:** trained on one trace, one pod mapping. Policy memorised a single datacenter's pattern.
- ▶ **After:** 7 training traces, random pod seed per episode, memory scale jitter $s \in [0.7, 1.2]$.
- ▶ Episodes are diverse in workload intensity and VM arrival patterns.

10 Azure traces (7 train / 2 test / 1 val)

Episode augmentation
scale s , pod seed, multi-trace

32 parallel environments
(SubprocVecEnv)

SAC (replay buffer 300k,
batch 512, $\gamma = 0.999$)

Eval: 50 random pod mappings
on held-out test traces

2. Throughput: 32 parallel environments

The reward was redesigned to score durable load, not raw load.

Old reward (run 6 — collapsed) New reward A (run 7 — stable)

Penalise increase in the *single worst MPD's* raw load.

- ▶ All other MPDs invisible — zero gradient signal.
- ▶ No departure model: 80 GB departing in 10 minutes looks the same as 80 GB staying for 3 days.
- ▶ **Result:** entropy collapsed at 100k steps; policy jammed all load onto one MPD.

Penalise the worst *projected sticky load* among all MPDs this host can reach.

- ▶ Sticky load = current load — memory leaving in ~ 17 hrs.
- ▶ Every reachable MPD generates signal every step.
- ▶ **Result:** entropy healthy throughout 2M steps; load spread evenly.

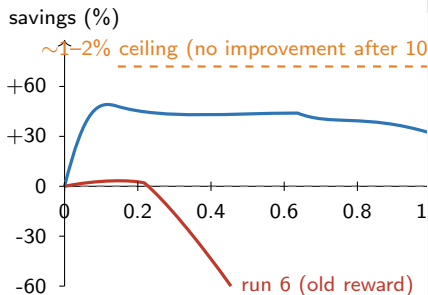
now

VM A, 40 GB

VM B, 60 GB

Raw load: 100 GB
Sticky load: ≈ 62 GB
Place here.

Savings plateau at $\sim 1\text{--}2\%$ from step 100k onward.



What we see

- ▶ Run 6 (old reward): policy collapses by 400k steps. Entropy hits $\alpha = 0.001$; savings dive to -62% .
- ▶ Run 7 (reward A): stable, doesn't collapse. Reaches $1\text{--}2\%$ within 40k steps and *stays there* for the full 2M steps.
- ▶ $2\text{M steps} \times 32 \text{ envs} \approx 64\text{M}$ environment interactions. More training is not the fix.

The plateau appears within the first 100k steps and does not improve

The agent is gaming the proxy reward, not maximising savings.

What we asked for

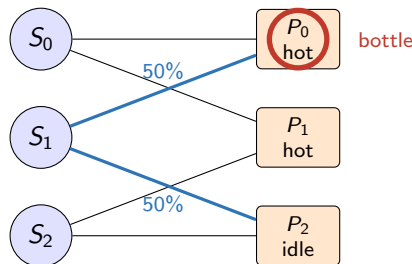
Minimise the worst *projected sticky load* among reachable MPDs. We hoped this would translate to low peak load over the full trace.

What the agent learned

Spread memory uniformly across reachable MPDs at every step. This minimises the per-step proxy reward perfectly — but it *ignores topology*: not all MPDs are equally shared, so uniform spread is not the same as globally balanced allocation.

The gap

The proxy reward (one step, one host's



Agent sees: S_1 's reachable MPDs look equal.
Reality: P_0 is shared by many hosts.

Two concrete fixes for tomorrow.

Fix 1: Optimal-gap reward shaping

We have an exact solver (`find_optimal`) that computes the best achievable peak for any snapshot of host demands using max-flow.

Add a penalty proportional to how far above optimal the agent's allocation sits:

$$r'_t = r_t^A - \beta \cdot \frac{C_{\max}^{\text{post}} - t^*(s_t)}{D_{\text{pod}}}$$

The agent now knows exactly how far from optimal it is at every step — not just whether the proxy improved.

Fix 2: Behavioral cloning warm-start

The solver also produces optimal allocation *weights* (flow solution → GB per MPD → normalised proportions).

Pre-train the actor on these (s, a^*) pairs before SAC starts. The policy begins near “instantaneously optimal” rather than random, preventing early entropy collapse.

SAC then fine-tunes from that warm start toward temporally optimal (accounts for VM lifetimes).